Web clients and servers have to follow a set of rules and procedures in order to successfully and correctly exchange data. These rules are more accurately known as *protocols*, which operate at various layers of your system (from network hardware up to the web application itself), and there are dozens of them involved in the process of requesting and interacting with web pages. For the internet, these rules are defined in a set of standards documents known as RFCs (Requests for Comments) by an organization called the IETF (Internet Engineering Task Force). RFCs describe in fine detail how systems should interoperate and, for the most part, boil down to when and what messages (usually in the form of strings of English text) should be sent over the network connection. For example, RFC 1945 – which can be found at `https://www.ietf.org/rfc/rfc1945.txt` – is the original standards document for HTTP (dating from 1996 and since obsoleted several times by more current standards). It's never a terrible idea to read RFCs, but it's rare that you'll need to know about any particular protocol in that level of detail – you'll be glad to know no such knowledge is assumed or required for this chapter. In this section, we'll examine in detail the protocol that you will encounter most when using PHP to build web applications.

The Hypertext Transfer Protocol is an application layer set of rules that enable the exchange of hypertext documents (and other related resource types) between systems via a lower-level (network) protocol (such as TCP/IP). HTTPS is simply HTTP with transport layer security (TLS) baked in to provide encryption to all the data exchanged between client and server; there is little functional difference as far as PHP is concerned. With HTTP, a *client* initiates a connection and then sends a *request message*. This message is then handled by another system acting as the *server*, which sends data back to the client in a *response message*. We won't go deep into the network side of things (you'll need another book for that); just know that after a bit of low-level negotiation between the client and server, a network connection is established between the two (by convention on TCP port 80 for HTTP and TCP 443 for HTTPS). Both request and response messages are simply a byte stream of ASCII text sequences that adhere to the protocol specification.

Please bear in mind that this section won't be an exhaustive reference for HTTP – we're just going to focus on the aspects commonly seen when processing web requests in PHP (e.g., you'll likely never need to worry about handling a `CONNECT` request in PHP; these are intended for controlling proxy servers). For the full details on HTTP (and other related technologies), I highly recommend looking at the Mozilla Foundation's excellent developer reference documents; the HTTP reference guide can be found at `https://developer.mozilla.org/en-US/docs/Web/HTTP`.